

Laudo de Virada Arquitetural: Pipeline de Markdown Estrutural como Fonte Única de Contexto

Projeto Assistente de Editais — Branch dev_rag_md — Abril 2026

1. Contexto

Os laudos 04 e 05 documentaram limitações remanescentes da arquitetura baseada em chunking de caracteres fixos sobre texto extraído via *pdfplumber*: contaminação entre ênfases em editais multi-área, ausência de rastreabilidade (página, seção), e fragmentação de conteúdo tabular. O laudo 05 investigou o Docling como possível camada de entrada estruturada e concluiu que a ferramenta recupera páginas e tabelas, mas não entrega hierarquia de seções utilizável, deixando o diagnóstico em aberto. Este laudo documenta a virada arquitetural executada a partir daí: substituição do *pdfplumber* por um pipeline de Markdown estruturado gerado por LLMs multimodais, reformulação do RAG para operar sobre chunks hierárquicos com metadados ricos, e um redesenho das ferramentas do agente com regras de *fallback* obrigatórias.

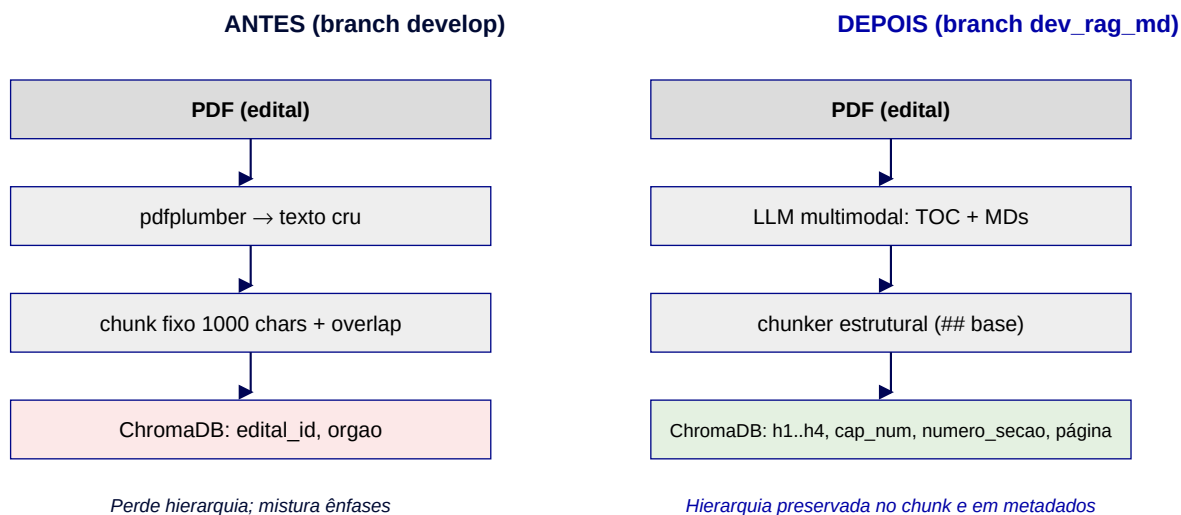


Figura 1. Comparação arquitetural. A representação canônica do edital passa de texto cru para Markdown hierárquico, e os metadados do RAG deixam de ser mínimos para refletir a estrutura original do documento.

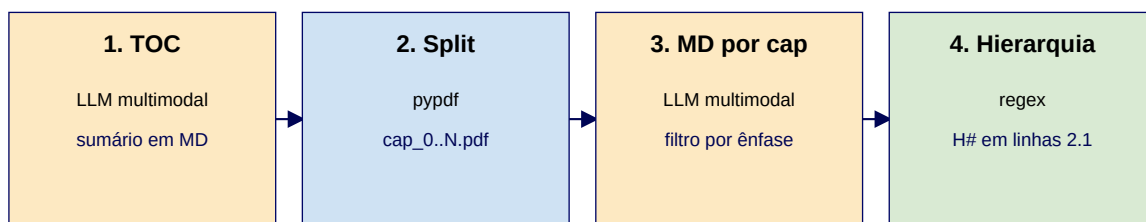
2. Decisão arquitetural

A decisão central foi trocar o problema: em vez de reconstruir hierarquia *post hoc* sobre texto achatado, passamos a gerar o corpus já em formato hierárquico — Markdown com headers #, ##, ### — antes da indexação. A geração é feita por LLMs multimodais que leem o PDF diretamente, aproveitando a capacidade desses modelos de distinguir cabeçalhos reais de parágrafos numerados — tarefa onde *pdfplumber* + regex falhava de forma não determinística (laudo 05, seção 3.2).

Com o Markdown como representação canônica do edital, três benefícios emergem simultaneamente: (1) a hierarquia passa a ser explícita e parseável, viabilizando *structure-aware chunking*; (2) a filtragem por ênfase pode ser aplicada na geração do MD via prompt, não por heurística posterior, resolvendo a contaminação na origem; (3) o mesmo corpus alimenta extração estruturada e RAG, eliminando a divergência entre duas fontes de texto do mesmo edital (laudo 03, seção 2).

3. O pipeline em quatro fases

O módulo `md_pipeline` opera em quatro fases, cada uma em arquivo separado e idempotente — se os artefatos de uma fase existem em disco, ela é pulada em reexecuções.



Cada fase gera artefatos em disco e é idempotente

Figura 2. Fases do md_pipeline.

3.1 Extração de TOC via LLM multimodal

O PDF inteiro é enviado em base64 para um modelo multimodal. Um prompt específico pede a estrutura útil do documento — cabeçalhos numerados até nível 4 e Anexos/Apêndices — com uma regra crítica que distingue cabeçalhos reais de parágrafos numerados. Essa distinção é a fraqueza clássica das abordagens por regex: editais usam numeração hierárquica tanto em títulos quanto em parágrafos de texto corrido. O prompt instrui explicitamente: *um item só é cabeçalho quando tem rótulo curto e estrutural, visualmente destacado; se após o número vier texto corrido ou frase completa, OMITA*. A saída é parseada para produzir `{editais}_toc.txt` com ranges de página, que funciona como contrato entre as fases seguintes.

3.2 Split físico por capítulo de nível 1

O TOC é parseado — apenas cabeçalhos de nível 1 — e o PDF é dividido fisicamente em `cap_0.pdf` (preâmbulo) até `cap_N.pdf`. Essa etapa é determinística (*pypdf*) e barata. Ranges de página podem invadir o início do capítulo seguinte devido a quebras de layout; essa imprecisão é resolvida na fase seguinte.

3.3 PDF de capítulo → Markdown com filtro por ênfase

Cada `cap_N.pdf` é enviado ao LLM multimodal com um prompt que executa três tarefas simultaneamente: (a) transcreve o conteúdo preservando texto literal, numeração e tabelas; (b) para quando encontra o título do próximo capítulo — passado como parâmetro no prompt — resolvendo a imprecisão do split; (c) aplica filtro por ênfase, omitindo seções exclusivas de outras áreas e preservando conteúdo comum e o específico de Ciência de Dados.

A separação de responsabilidades é importante: o LLM multimodal é bom em três tarefas distintas — identificar estrutura (fase 1), transcrever conteúdo (fase 3) e filtrar por critério semântico (fase 3). Cada uma é atacada com um prompt dedicado, sem mistura. O `cap_0` recebe tratamento especial: como é preâmbulo e não tem hierarquia, o prompt pede texto corrido sem formatação Markdown.

3.4 Pós-processamento da hierarquia

Em inspeção, observou-se que o LLM ocasionalmente devolve o conteúdo com a numeração correta (2.1 Texto...) mas sem marcar a linha como header Markdown. Um passo determinístico aplica regex sobre linhas iniciadas por numeração hierárquica e converte-as em headers do nível correspondente. `Cap_0` e capítulos de Anexo/Apêndice são excluídos dessa etapa — o primeiro porque é texto corrido, os segundos porque a numeração interna de anexos não segue a hierarquia do corpo principal.

4. Structure-aware chunking

Com os Markdowns gerados e hierarquia garantida, o chunker para o ChromaDB foi reescrito. A unidade base passa a ser a seção `##`. O processo combina três operações:

Descarte de H1 órfãos. Um capítulo cujo H1 tem conteúdo vazio (todo o texto está em `##` filhos) tem seu H1 descartado como chunk; o título persiste apenas como contexto nos filhos. Isso elimina ruído — inspeção inicial mostrou chunks de 17 caracteres contendo apenas `# 9. DAS PROVAS`, sem valor para retrieval.

Merge de blocos pequenos. Blocos abaixo de 400 caracteres são fundidos com irmão vizinho sob o mesmo H1, respeitando um teto de 3500 caracteres. O ponto sutil — que exigiu uma segunda iteração — é que, ao fundir, o header do segundo bloco é preservado dentro do conteúdo fundido. A primeira versão concatenava apenas texto e perdia `#### 1 - LÍNGUA PORTUGUESA` no meio do chunk, tornando o Markdown inválido e desorientando o embedding.

Split de blocos grandes. Blocos acima de 3500 caracteres são subdivididos primeiro por sub-headers internos (`####`), e depois por parágrafo até ~1500 caracteres quando necessário. Sub-chunks herdam os headers ancestrais do pai.

Cada chunk final é um pedaço de Markdown válido com a cadeia de ancestrais reinjetada no topo. Exemplo real (CVM, capítulo 16):

```
# ANEXO I - CONTEÚDO PROGRAMÁTICO
## CONHECIMENTOS ESPECÍFICOS: ANALISTA - Ciência de Dados - (Perfil 7)
#### 2 - LINGUAGENS E BANCO DE DADOS

Linguagem de programação Python. Sintaxe básica, operadores, variáveis. Estruturas de dados [...]

```

A diferença em relação a um *breadcrumb* textual (Contexto: `X > Y > Z`) é relevante: o chunk permanece como Markdown válido, o embedding recebe os headers com seus marcadores sintáticos, e a exibição ao usuário preserva a estrutura. Cada chunk carrega metadados ricos: `edital_id`, `orgao`, `cap_num`, `cap_titulo`, `tipo`, `h1`, `h2`, `h3`, `h4`, `numero_secao`, `pagina_inicio`, `pagina_fim`, `chunk_index`.

5. Reformulação do agente

O conjunto de ferramentas do agente foi reorganizado em torno da nova representação, e o procedimento operacional padrão (SOP) reescrito. A tabela abaixo resume as mudanças.

Ferramenta	Status	Descrição
listar_editais	Mantida	Retorna IDs e metadados dos editais cadastrados. Primeiro passo obrigatório do SOP.
consultar_edital	Mantida	Consulta campos estruturados do JSON (salário, jornada, vagas agregadas, etc.) por <code>edital_id</code> .
buscar_no_edital	Reescrita	<i>top-k</i> default aumentado de 5 para 12. Agrupa trechos por capítulo, ordena por <code>chunk_index</code> dentro do grupo. <i>Fallback</i> automático para busca sem filtro quando o filtro por <code>edital_id</code> retorna poucos resultados.
resumo_edital	Mantida	Retorna o resumo pré-gerado do edital, focado em Ciência de Dados.
listar_capitulos	Nova	Expõe o sumário do edital (TOC) para que o agente escolha o <code>cap_num</code> correto antes de ler um capítulo inteiro.

ler_capitulo	Nova	Devolve o cap_N.md inteiro. Perguntas que cobrem uma seção completa — conteúdo programático, cronograma, regras gerais das provas — são mal atendidas por similaridade vetorial. Com os Markdowns em disco, ler o capítulo direto é operação trivial.
data_hoje	Nova	Retorna a data atual. O agente é instruído a chamá-la sempre que a pergunta envolver vigência, prazos ou comparação com datas do cronograma. Força a comparação a virar um ato explícito do raciocínio.

Duas regras foram incorporadas ao SOP e são as mudanças de comportamento mais visíveis:

Comparação de datas obrigatória. Para qualquer pergunta sobre vigência, prazos ou se algo “está aberto”, o agente precisa chamar `data_hoje()` antes de comparar com as datas do cronograma. Embora a data atual já estivesse disponível no system prompt, observou-se em LangSmith que o LLM frequentemente deixava de fazer a comparação explícita — afirmava que as inscrições ocorreriam em uma data futura sem verificar se essa data já havia passado. Transformar a data em resultado de ferramenta evita essa omissão.

Fallback obrigatório. Nenhuma ferramenta retornando resultado vazio ou parcial justifica responder “não encontrei”. O agente é obrigado a tentar uma ferramenta alternativa — tipicamente `buscar_no_edital` ou `ler_capitulo` — antes de declarar ausência. Esse padrão foi identificado como *faithfulness hallucination* no laudo 03 (Huang et al., 2025): o modelo afirmando não ter informação que existe no corpus mas não foi acessada pela rota escolhida. Caso observado na avaliação: a pergunta sobre vagas PcD foi respondida com “não há menção específica” porque a ferramenta `consultar_edital` devolveu campos agregados, e o agente não tentou o RAG. Com a regra de fallback ativa, essa classe de erro é quebrada.

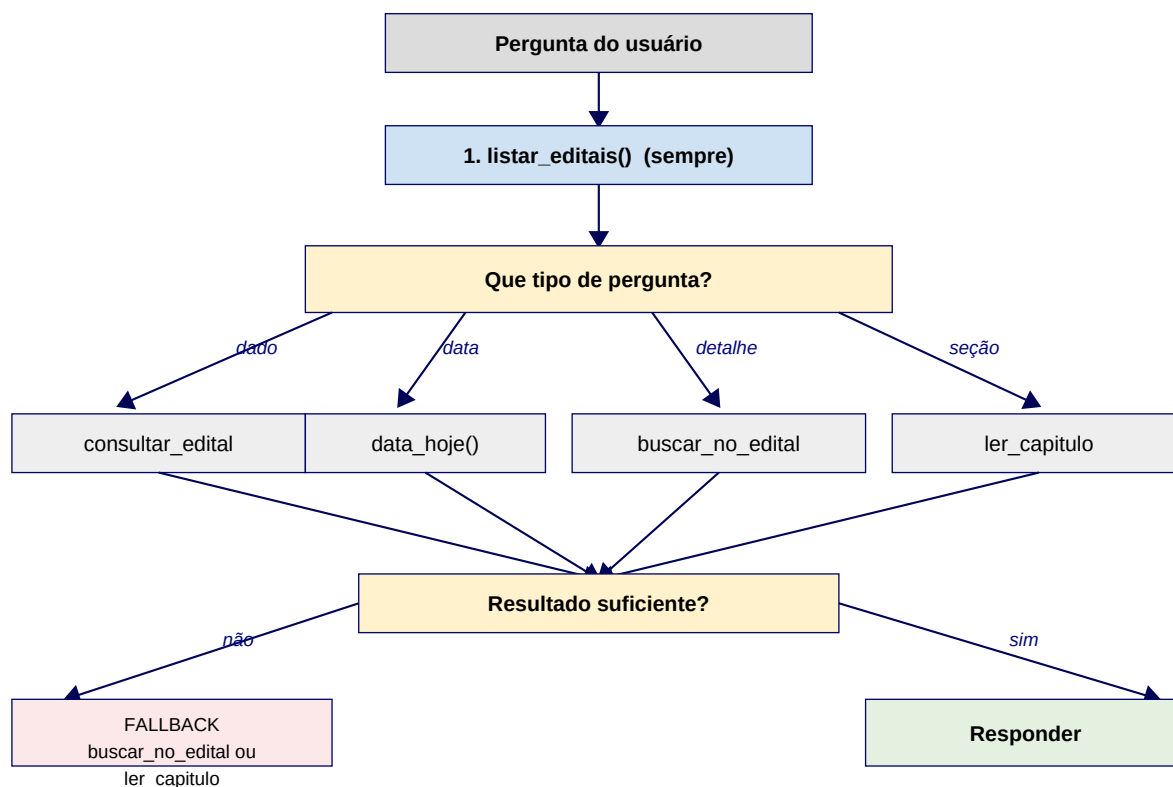


Figura 3. Fluxo de decisão do agente. Primeiro passo obrigatório é `listar_editais()`. A rota depende do tipo de pergunta; resultados insuficientes disparam fallback antes de o agente responder.

6. Como os problemas anteriores ficam atacados

A tabela abaixo cruza os problemas dos laudos 03 a 05 com o tratamento aplicado nesta virada.

Problema (laudo de origem)	Tratamento nesta arquitetura
Contaminação entre ênfases (05)	Filtro aplicado no prompt de geração do MD (fase 3.3). Conteúdo de outras ênfases nunca entra no corpus. Chunk carrega <i>h1..h4</i> como metadata filtrável.
Ausência de rastreabilidade (05)	Cada chunk carrega <i>pagina_inicio</i> , <i>pagina_fim</i> , <i>cap_num</i> , <i>numero_secao</i> e <i>h1..h4</i> . Resposta pode citar <i>item 9.6.1, p. 12</i> .
Conteúdo tabular fragmentado (05)	LLM multimodal transcreve tabelas em sintaxe Markdown na fase 3.3. Tabela permanece íntegra dentro do chunk da seção que a contém.
Hierarquia não reconstrutível (05)	Hierarquia é produzida pelo LLM no TOC (fase 3.1), validada pela regex de pós-processamento (3.4), e preservada dentro de cada chunk do RAG.
Agente desiste na primeira falha (03)	Regra de fallback obrigatório no SOP. Ferramentas sem resultado disparam tentativa por rota alternativa antes de qualquer resposta de ausência.
Lookup frágil por nome de órgão (03)	Substituído por <i>edital_id</i> numérico em todas as ferramentas. O nome do órgão foi adicionalmente normalizado para sigla curta (CVM, BNDES, Petrobras) na extração estruturada.
Comparação de data não executada (03)	Ferramenta <i>data_hoje()</i> e regra explícita no SOP forçam a comparação a ser um ato do raciocínio.

As três restrições de projeto do laudo 05 permanecem respeitadas. A solução não enviesa para o formato BNDES (5.1): o prompt de TOC trabalha sobre critérios genéricos — cabeçalho curto vs parágrafo corrido — aplicáveis a qualquer edital. O filtro por ênfase é declarado via parametrização do prompt, não por nome fixo de seção. O pipeline é automático (5.2): nenhuma intervenção manual entre submissão do PDF e chunks indexados. Metadados filtráveis são armazenados em formato consistente e não dependem de comparação sensível a acento (5.3).

7. Custo e intercambialidade de modelos

O pipeline depende de duas famílias de chamadas multimodais por edital: uma para o TOC e N-1 para os capítulos. Durante desenvolvimento, ambas as fases usam Gemini 3.1 Flash-Lite como modelo padrão, minimizando custo de iteração. As constantes `VAR_TOC` e `VAR_MD` em `md_pipeline/config.py` permitem trocar para o cenário ideal — Claude Sonnet 4.5 para TOC (maior robustez na distinção cabeçalho/parágrafo) e Claude Haiku 4.5 para os capítulos — em duas linhas de código. A camada `llm_multimodal.py` centraliza a diferença entre providers: Anthropic usa blocos de tipo *document*, Google usa blocos de tipo *media*.

Como o pipeline é idempotente, o custo se paga uma vez por edital. Reingerir um edital já processado (por exemplo, após limpar o ChromaDB) não dispara novas chamadas multimodais: os Markdowns em disco são reaproveitados, e apenas a indexação vetorial e a extração estruturada sobre o texto-base concatenado são refeitas.

8. Avaliação empírica

A virada foi implantada em duas rodadas de correção. A primeira consolidou o pipeline de Markdown e o chunker estrutural, e foi avaliada com a mesma bateria do laudo 04 sobre o edital BNDES: oito acertos

plenos, um acerto parcial e um erro, totalizando 8,5/10. A tabela abaixo registra esse resultado e identifica as duas correções aplicadas em seguida, motivadas pela análise das falhas observadas.

Questão	1ª rodada	Falha observada / correção aplicada
Salário	Correto	—
Vagas PcD	Incorreto	Agente respondeu “não há menção” após <i>consultar_edital</i> devolver campos agregados. Correção: regra de fallback obrigatório no SOP força <i>buscar_no_edital</i> antes de qualquer resposta de ausência.
Requisito	Correto	—
Inglês	Correto	—
Conteúdo (13 blocos)	Correto	—
Nota mínima	Correto	—
Relógio	Correto	—
Inscrições	Meio correto	Agente apresentou o intervalo de datas sem comparar com o dia atual. Correção: ferramenta <code>data_hoje()</code> e regra no SOP forçam a comparação explícita.
Jornada	Correto	—
Desempate	Correto	—

Uma observação sobre protocolo de avaliação merece registro. A avaliação inicial automatizada (ChatGPT 5.4 atuando como juiz) classificou **incorretamente** respostas sobre inglês e nota mínima, conteúdo que estava correto segundo o item 8.1.3 do edital. Revisão manual mostrou que o erro foi do avaliador, não do sistema. A causa técnica é o fenômeno de **excesso de contexto**: o modelo avaliador ancorou-se em conhecimento geral sobre concursos em vez do trecho específico do edital, enquanto o sistema RAG, restrito ao contexto recuperado, manteve-se fiel ao documento. Em tarefas de validação, essa assimetria favorece sistemas recuperativos.

Com as correções da segunda rodada (fallback obrigatório e tool de data), as duas falhas identificadas deixam de ser estruturais e passam a depender apenas de o agente seguir o SOP — comportamento observável em LangSmith. Uma reaplicação da bateria está prevista e será documentada em laudo próprio.

9. Estado atual e próximos passos

A branch `dev_rag_md` consolida: (i) pipeline `md_pipeline` em quatro fases, idempotente, com providers de LLM multimodal configuráveis; (ii) chunker estrutural com metadados ricos; (iii) RAG v2 sobre ChromaDB preservando a API existente; (iv) conjunto ampliado de ferramentas do agente, incluindo leitura direta de capítulo e tool de data; (v) SOP reescrito com regras explícitas de comparação de data e fallback obrigatório. Os três editais de teste (BNDES, CVM, Petrobras) foram reingeridos sobre a nova arquitetura aproveitando os Markdowns pré-existentis, sem novas chamadas de LLM multimodal.

Direções identificadas para evolução: (a) nova bateria de avaliação pós-correções, em três editais; (b) aproveitamento dos metadados de seção para filtros mais precisos no retrieval (por exemplo, *tipo=anexo* para perguntas sobre conteúdo programático); (c) reranking por proximidade estrutural — direção B do laudo 05 —, agora trivial com `chunk_index` disponível.

10. Referências

Huang, L. et al. (2025). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(5), 1-55. DOI: 10.1145/3703155.

Liu, N. F. et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157-173. DOI: 10.1162/tacl_a_00638.